MTRSP 自适应大邻域搜索算法补充材料

1 算法详细伪代码

1.1 任务优先级计算

Algorithm 1 计算任务优先级

Require: 任务集合 S，参数 (w_1, w_2, w_3)

Ensure: 优先级分数 $\text{Priority}(s), \forall s \in S$

```
for s = 1 to |S| do
    tw_width ← b_s − a_s {时间窗宽度}
    Urgency(s) ← 1/(tw_width+0.1) {紧迫度}
    Efficiency(s) ← H_s^min/(T_s+0.1) {效率}
    Complexity(s) ← 1/Γ_s {复杂度}
    Priority(s) ← w_1 · Urgency(s) + w_2 · Efficiency(s) + w_3 · Complexity(s)
end for
return Priority(s), ∀s ∈ S
```

1.2 可行拖船组合枚举

Algorithm 2 寻找可行拖船组合

Require: 任务 s，拖船集合 K

Ensure: 可行组合集合 $\mathcal{C}_s$

```
C_s ← ∅
for γ = 1 to min(Γ_s, |K|) do
    {组合大小}
    for 每个大小为 γ 的拖船组合 C ⊆ K do
        total_hp ← Σ_{k∈C} HP_k
        if total_hp ≥ H_s^min then
            {满足马力需求}
            C_s ← C_s ∪ {C}
        end if
    end for
end for
按照 (|C|, −Σ_{k∈C} HP_k) 排序 C_s {优先少而强的拖船}
return C_s 的前 20 个组合
```

1.3 计算最早开始时间

Algorithm 3 计算任务最早可行开始时间

Require: 当前解 $\mathcal{S}$，任务 s，拖船组合 C，插入位置 $\{p_k\}_{k\in C}$

Ensure: 最早开始时间 $\tau_s^{\min}$ 或 INFEASIBLE

earliest_arrival $\leftarrow 0$
for 每个拖船 $k \in C$ **do**
  if $p_k = 0$ **then**
    {路径的第一个任务}
    $\text{arrival}_k \leftarrow t_{0,s}$ {从仓库出发}
  else
    $s_{\text{prev}} \leftarrow$ 路径 k 中位置 $p_k - 1$ 的任务
    $\tau_{\text{prev}} \leftarrow$ 任务 s_{prev} 的开始时间
    $\text{arrival}_k \leftarrow \tau_{\text{prev}} + T_{s_{\text{prev}}} + t_{s_{\text{prev}},s}$
  end if
  earliest_arrival $\leftarrow \max(\text{earliest_arrival}, \text{arrival}_k)$
end for
$\tau_s^{\min} \leftarrow \max(\text{earliest_arrival}, a_s)$ {考虑时间窗下界}
if $\tau_s^{\min} > b_s$ **then**
  {超过时间窗上界}
  return INFEASIBLE
end if
if $\tau_s^{\min} + T_s > T^{\max}$ **then**
  {超过规划期}
  return INFEASIBLE
end if
return $\tau_s^{\min}$

1.4 插入成本计算

Algorithm 4 计算任务插入成本

Require: 任务 s，拖船组合 C，插入位置 $\{p_k\}_{k \in C}$，开始时间 τ_s

Ensure: 插入成本 ΔF

```
ΔF ← 0
for 每个拖船 k ∈ C do
    // 服务燃油
    service_fuel ← α_k · HP_k · T_s
    ΔF ← ΔF + service_fuel

    // 行驶燃油
    if p_k = 0 then
        prev_node ← 0 {仓库}
    else
        prev_node ← 路径 k 中位置 p_k − 1 的任务
    end if
    travel_time ← t_{prev_node,s}
    travel_fuel ← β_k · HP_k · travel_time
    ΔF ← ΔF + travel_fuel
end for
return ΔF
```

1.5 轮盘赌算子选择

Algorithm 5 基于权重的轮盘赌选择

Require: 算子权重向量 $\mathbf{w} = (w_1, w_2, \ldots, w_n)$

Ensure: 选中的算子索引 i^*

```
total ← Σ_{i=1}^{n} w_i
if total = 0 then
    return Uniform(1, n) {均匀随机选择}
end if
r ← Uniform(0, total)
cumsum ← 0
for i = 1 to n do
    cumsum ← cumsum + w_i
    if cumsum ≥ r then
        return i
    end if
end for
return n {容错返回}
```

2 破坏算子详细说明

2.1 最差移除算子 (d_2)

Algorithm 6 最差移除算子

Require: 当前解 $\mathcal{S}$，移除数量 q

Ensure: 部分解 $\mathcal{S}'$

```
task_costs ← ∅
for 每个已调度任务 s ∈ 𝒯 do
    cost ← 0
    for 每个服务任务 s 的拖船 k ∈ K_s do
        // 服务成本
        cost ← cost + α_k · HP_k · T_s

        // 行驶成本
        i ← 任务 s 在路径 k 中的前驱
        j ← 任务 s 在路径 k 中的后继
        cost ← cost + β_k · HP_k · (t_{i,s} + t_{s,j})
    end for
    task_costs[s] ← cost
end for
按照 cost 降序排序 tasks
移除成本最高的 q 个任务
return 部分解 S'
```

2.2 Shaw 移除算子 (d_3)

Algorithm 7 Shaw 移除算子（相关性移除）

Require: 当前解 $\mathcal{S}$，移除数量 q

Ensure: 部分解 $\mathcal{S}'$

```
随机选择种子任务 s_0 ∈ T
tw_center_0 ← (a_{s_0} + b_{s_0}) / 2 {时间窗中心}
relatedness ← ∅
for 每个已调度任务 s ∈ T \ {s_0} do
    tw_center_s ← (a_s + b_s) / 2
    // 时间距离
    time_dist ← |tw_center_s − tw_center_0|
    // 空间距离（用行驶时间近似）
    space_dist ← t_{s_0,s}
    relatedness[s] ← time_dist + space_dist
end for
relatedness[s_0] ← ∞ {种子任务必须移除}
按照 relatedness 升序排序 tasks
移除相关度最高的 q 个任务
return 部分解 S'
```

3 修复算子详细说明

3.1 Regret-2 插入算子 (r_2)

Algorithm 8 Regret-2 插入算子

Require: 部分解 $\mathcal{S}'$，未调度任务集合 $\mathcal{U}$

Ensure: 完整解 $\mathcal{S}$

```
S ← S'
while U ≠ ∅ do
    best_regret ← −∞
    s* ← null, insertion* ← null
    for 每个未调度任务 s ∈ U do
        costs ← [] {存储所有可行插入的成本}
        for 每个可行拖船组合 C ∈ C_s do
            for 每个可行插入位置组合 p do
                if 插入 (C, p) 可行 then
                    ΔF ← 计算插入成本
                    costs.append(ΔF)
                end if
            end for
        end for
        if |costs| ≥ 2 then
            对 costs 排序：ΔF_1 ≤ ΔF_2 ≤ ···
            regret ← ΔF_2 − ΔF_1 {后悔值}
        else if |costs| = 1 then
            regret ← 10000 {只有一个选择，后悔值很大}
        else
            continue {无可行插入}
        end if
        if regret > best_regret then
            best_regret ← regret
            s* ← s
            insertion* ← 成本最小的插入方式
        end if
    end for
    if s* ≠ null then
        执行 insertion*，将 s* 插入到 S
        U ← U \ {s*}
    else
        break {没有更多可行插入}
    end if
end while
```

4 局部搜索算子

4.1 任务交换算子

Algorithm 9 任务交换局部搜索

Require: 当前解 $\mathcal{S}$

Ensure: 改进解 $\mathcal{S}'$

```
S^best ← S
f^best ← f(S)
for 拖船 k_1 = 1 to |K| − 1 do
    for 拖船 k_2 = k_1 + 1 to |K| do
        for k_1 路径中的每个任务 s_i (位置 p_1) do
            for k_2 路径中的每个任务 s_j (位置 p_2) do
                S' ← 交换 s_i 和 s_j 的分配
                重新计算 S' 中所有任务的开始时间
                if S' 可行 and f(S') < f^best then
                    S^best ← S'
                    f^best ← f(S')
                end if
            end for
        end for
    end for
end for
return S^best
```

4.2 2-opt 算子

Algorithm 10 2-opt 路径优化

Require: 当前解 $\mathcal{S}$

Ensure: 改进解 $\mathcal{S}'$

```
S^best ← S
f^best ← f(S)
for 每个拖船 k ∈ K do
    route ← 拖船 k 的路径
    if |route| < 2 then
        continue
    end if
    for i = 0 to |route| − 2 do
        for j = i + 2 to |route| do
            // 反转 route[i:j]
            route' ← route[0 : i] + reverse(route[i : j]) + route[j :]
            S' ← 用 route' 替换拖船 k 的路径
            重新计算时间
            if S' 可行 and f(S') < f^best then
                S^best ← S'
                f^best ← f(S')
            end if
        end for
    end for
end for
return S^best
```

5 算法流程图

5.1 ALNS 主流程

6 参数敏感性分析建议

在论文中，建议进行以下参数敏感性分析：

6.1 破坏率 q

测试不同的破坏率范围：

- $[q_{\min}, q_{\max}] \in \{[0.05, 0.2], [0.1, 0.3], [0.1, 0.4], [0.2, 0.5]\}$

  预期：适中的破坏率（10-40%）效果最好。

6.2 温度参数

测试不同的初始温度和冷却率：

- $T_0 \in \{500, 1000, 2000, 5000\}$
- $\alpha \in \{0.99, 0.995, 0.998, 0.999\}$

  预期：较高的初始温度和较慢的冷却率提供更好的探索能力。

6.3 奖励参数

测试不同的奖励值：

- $(\sigma_1, \sigma_2, \sigma_3) \in \{(10, 5, 1), (33, 9, 3), (100, 20, 5)\}$

  预期：奖励比例对算子选择的影响显著。

6.4 迭代次数

测试不同的最大迭代次数：

- $I_{\max} \in \{1000, 2500, 5000, 10000, 20000\}$

  分析：绘制目标函数值随迭代次数的收敛曲线。

7 结果展示建议

7.1 性能比较表格模板

7.2 算子使用频率表格模板

7.3 收敛曲线建议

建议绘制以下图表：

1. 目标函数值随迭代次数的变化曲线
2. 当前解和最优解的演化曲线
3. 温度下降曲线
4. 算子权重随迭代的演化

表 1: 算法性能比较

2* 实例	2*$\|S\|$	2*$\|K\|$	Gurobi		ALNS		
			目标值	时间 (s)	目标值	Gap(%)	时间 (s)
E_10_5_v1	10	5	28,450.23	15.2	29,964.87	5.3	1.5
E_10_5_v2	10	5	-	-	-	-	-
E_20_10_v1	20	10	-	-	-	-	-
平均	-	-	-	-	-	-	-

表 2: 算子使用统计

算子	调用次数	成功次数	成功率 (%)	最终权重
破坏算子				
Random Removal	1,245	156	12.5	1.23
Worst Removal	1,189	203	17.1	1.45
Shaw Removal	1,312	178	13.6	1.18
Tugboat Removal	987	145	14.7	1.34
Time-based Removal	1,267	189	14.9	1.29
修复算子				
Greedy Insertion	1,534	312	20.3	1.67
Regret-2 Insertion	1,423	289	20.3	1.58
Time-oriented Insertion	1,211	187	15.4	1.12
Cluster Insertion	832	83	10.0	0.87

8 论文写作建议

8.1 算法描述结构

建议按以下结构组织算法描述章节：

1. **引言**：介绍 ALNS 的优势和适用性
2. **算法框架**：给出主算法伪代码和流程图
3. **初始解构造**：详细描述任务优先级和贪心插入
4. **破坏算子**：逐一介绍 5 种破坏算子及其设计动机
5. **修复算子**：逐一介绍 4 种修复算子及其特点
6. **自适应机制**：解释权重更新和算子选择机制

7. **局部搜索**：介绍后处理优化方法
8. **参数设置**：说明参数选择的依据
9. **复杂度分析**：分析时间和空间复杂度

8.2 关键创新点强调

在论文中应强调以下创新点：

1. **问题特定的算子设计**：
   - Tugboat-based removal 算子专门针对多拖船协同问题
   - 考虑燃油容量约束的插入成本计算
2. **多拖船协同任务处理**：
   - 统一处理单拖船和多拖船任务
   - 维护路径分配的一致性
3. **任务优先级机制**：
   - 综合时间、效率和复杂度的多维评估
   - 引导初始解快速构造高质量解

8.3 实验设计建议

1. **实例规模**：
   - 小规模：10-20 任务，5-10 拖船
   - 中规模：50-100 任务，20-30 拖船
   - 大规模：150-200 任务，40-50 拖船
2. **比较基准**：
   - Gurobi 精确求解（小规模实例）
   - 简单贪心算法
   - 标准 ALNS（无问题特定算子）
   - 其他元启发式（GA、TS 等）
3. **评价指标**：
   - 目标函数值

- 任务完成率
- 计算时间
- 解的质量差距（与下界或最优解比较）
- 鲁棒性（多次运行的标准差）

```
    开始
     |
     v
初始解构造
     |
     v
+-------------------+
|   ALNS主循环      |
|   (迭代I_max次)   |
+-------------------+
     |
     +--> 选择破坏算子（轮盘赌）
     |
     +--> 执行破坏（移除q%任务）
     |
     +--> 选择修复算子（轮盘赌）
     |
     +--> 执行修复（重新插入任务）
     |
     +--> 评估新解
     |
     +--> 接受准则判断
     |    |
     |    +--> 新最优解? --> 更新最优解 + 权重+=
     |    |
     |    +--> 改进解? --> 更新当前解 + 权重+=
     |    |
     |    +--> 模拟退火接受? --> 更新当前解 + 权重+=
     |
     +--> 更新温度 T =  ·T
     |
     +--> 每100次迭代归一化权重
     |
     v
   局部搜索
     |
     v
   返回最优解
     |
     v
    结束
```